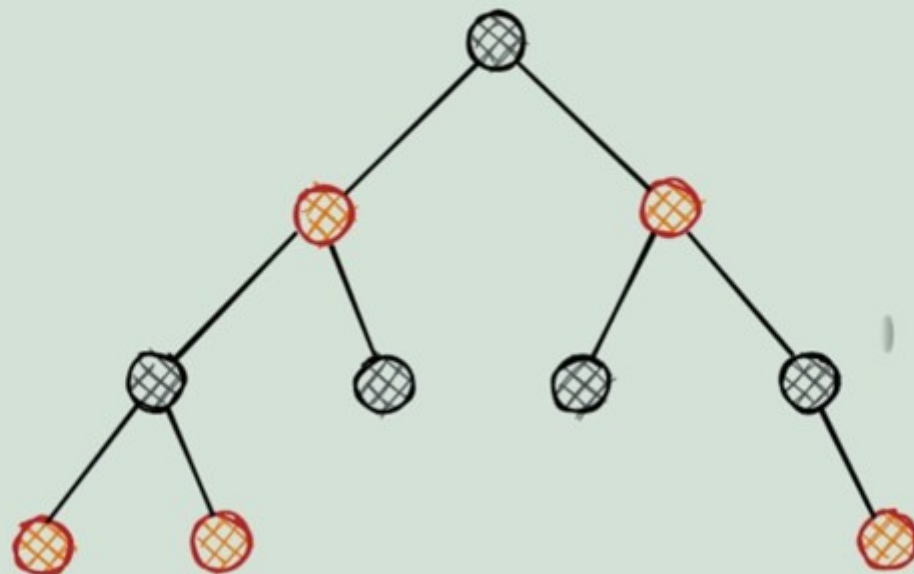


Red Black Tree Case Study

What is a Red-Black Tree?

- • A Red-Black Tree (RBT) is a **self-balancing Binary Search Tree (BST)**.
- • Each node has an extra bit for **color (Red or Black)**.
- • Ensures operations like insertion, deletion, and search take **$O(\log n)$** time.
- • Maintains approximate balance using **color properties and rotations**.

Red Black Tree



Need for Balancing

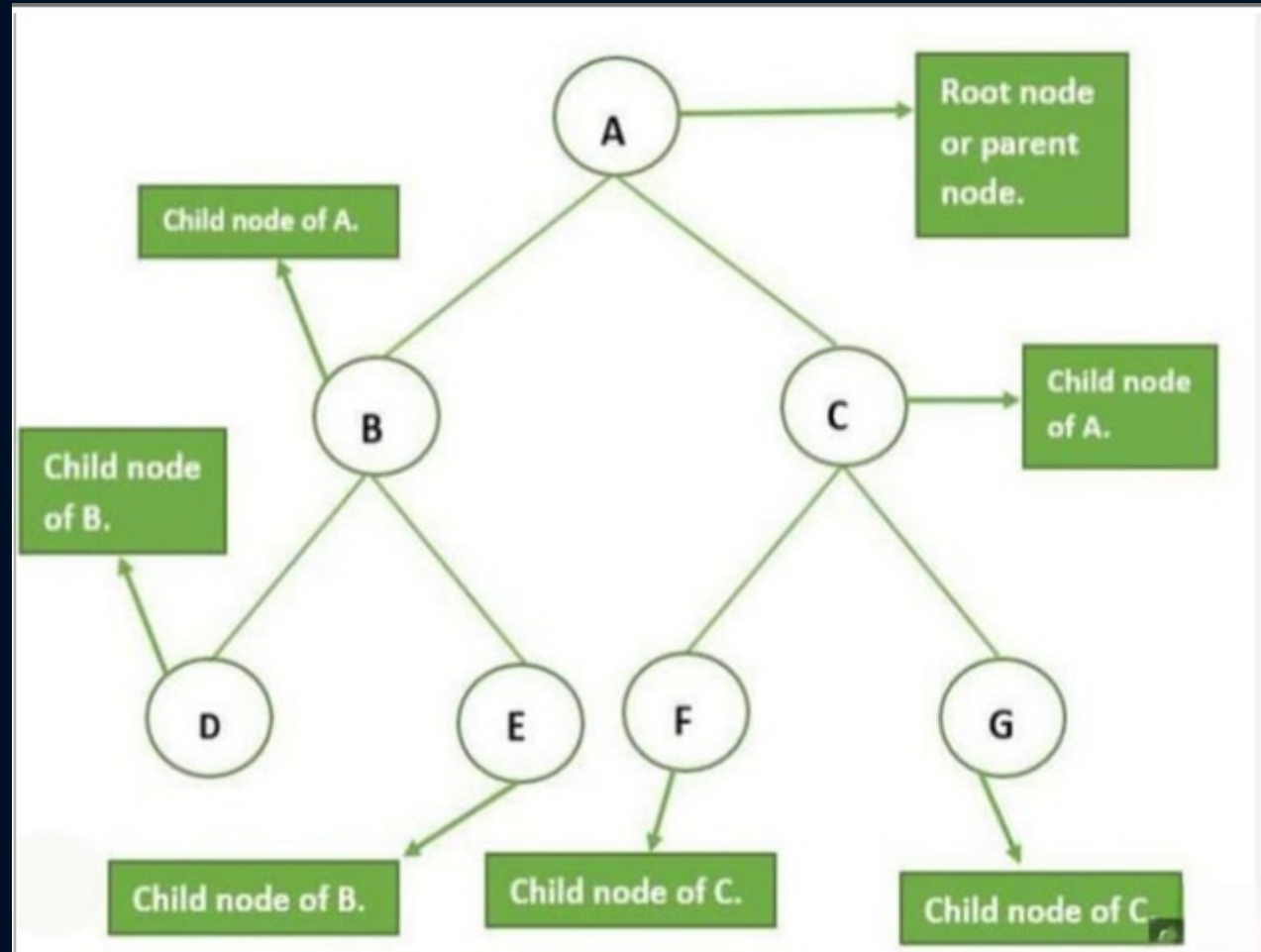
- Normal BST can become **unbalanced** (like a linked list).
- • Causes poor performance — **$O(n)$** time complexity.
- • RBT automatically adjusts structure to remain balanced

Comparison with AVL Tree

Aspect	Red-Black Tree (RBT)	AVL Tree
Balancing	Loosely balanced	Strictly balanced
Rotations	Fewer	More
Insertion/Deletion	Faster	Slower
Search	Slightly slower	Faster
Use case	Frequent updates	Search-heavy tasks

NODES STRUCTURE'S

- Each node in a Red-Black Tree stores data and extra information to help the tree stay balanced.
- **Key / Value** → The actual data stored in the node
- **Color Bit** → Indicates whether the node is **Red** or **Black**
- **Left Child Pointer** → Points to the left subtree
- **Right Child Pointer** → Points to the right subtree
- **Parent Pointer** → Helps navigate upward during rotations and fixes



Key Properties Of **Red** Black Tree

- A **Red - Black Tree** is a type of self-balancing binary search tree that follows specific color rules to maintain balance after insertions and deletions.
- Every node is either Red or Black.
- The Root node is always Black.
- All NIL (leaf) nodes are Black.
- If a node is Red, then both its children are Black (no two reds in a row).
- Every path from a node to its descendant NIL nodes has the same number of Black nodes (called Black Height).

Black Height Concept

- **Definition:**

- Black Height = Number of black nodes on any path from a node to a NIL (leaf) node.

- **Balance Indicator:**

Every path from the root to NIL has the same black height, ensuring uniform depth.

- **Why It Matters:**

- This rule keeps the tree approximately balanced, giving efficient operations in $O(\log_2 n)$ time.

- **Visual Intuition:**

- Imagine counting black beads along each root-to-leaf path — each path must have the same number, keeping the tree steady and fair!

Validity of Red – Black Tree

Valid RBT

- Ensures efficient and balanced search performance
- No two consecutive red nodes
- Equal **black height** on all root-to-leaf paths

In Valid RBT

- Two red nodes in a row (violates property 4)
- Unequal black height across paths (violates property 5)
- Root not black, or leaf NILs not black

Red-Black Tree Operations

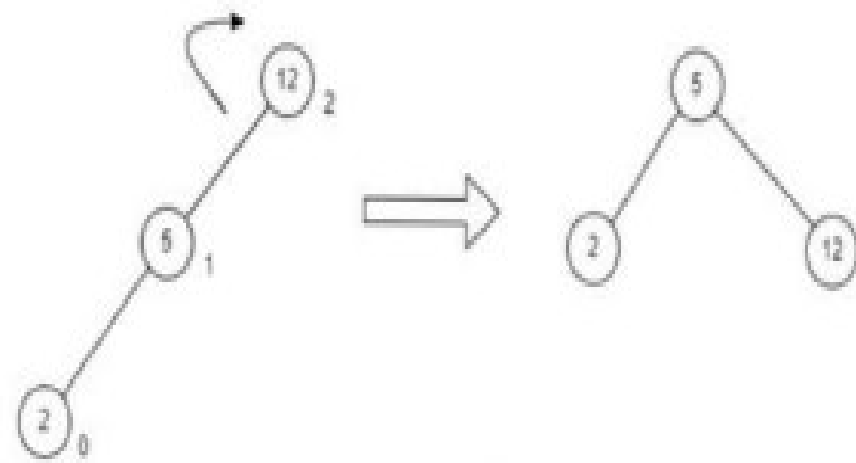
- Search:
- Works like a Binary Search Tree (BST).
- Example: Traverses left or right until found or null.
- - Complexity: $O(\log n)$

- 2. Insert:
- Insert new node like BST, color it red.
- Fix violations using recoloring or rotations.
- Complexity: $O(\log n)$

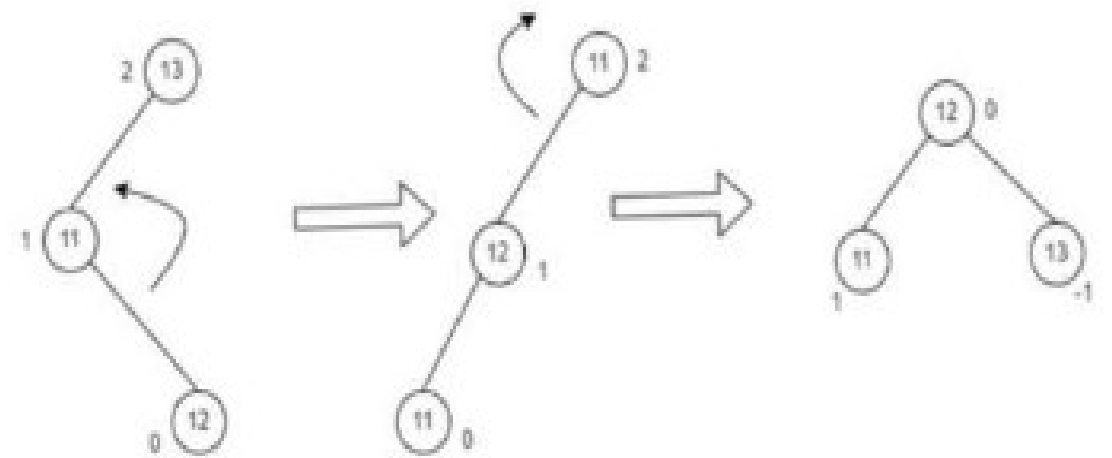
- 3. Delete:
- Perform standard BST delete.
- Fix double-black or color violations using

Tree Rotations in Red-Black Trees

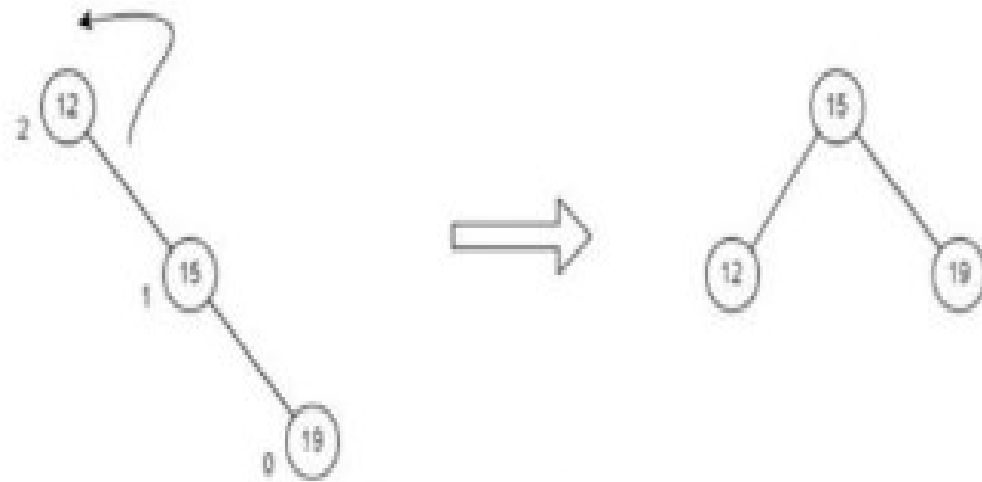
- Explain how left and right rotations help maintain the Red-Black Tree balance.
- 1. Left Rotation:
 - Used when a right-heavy subtree causes imbalance.
 - The parent node moves left, and the right child becomes the new parent.
- 2. Right Rotation:
 - Used when a left-heavy subtree causes imbalance.
 - The parent node moves right, and the left child becomes the new parent.
- How Rotations Maintain BST Property:
 - They rearrange the tree locally without changing the in-order sequence of nodes.
 - Hence, the Binary Search Tree (BST) property remains intact.
- Note:
 - Rotations are the building blocks of rebalancing during insertion and deletion



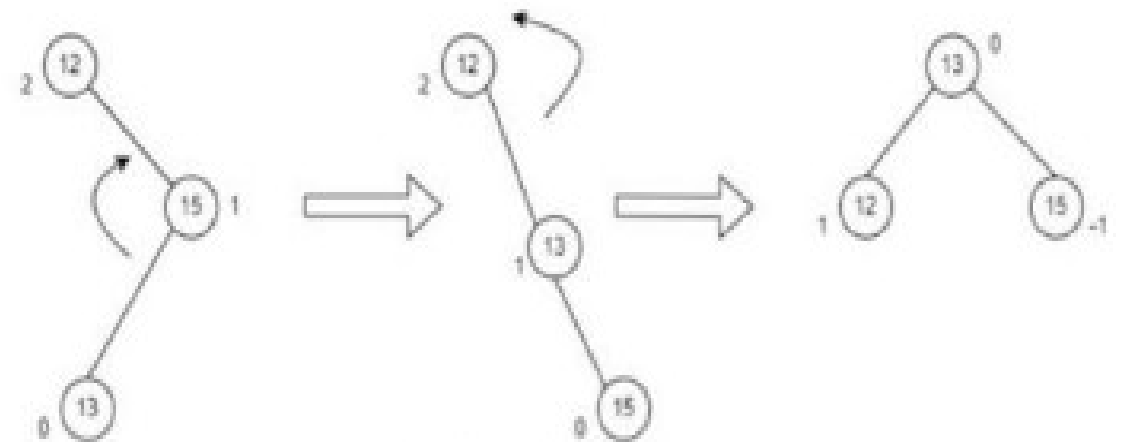
LL rotation



LR rotation



RR rotation



RL rotation

Fixing Violations in Red-Black Trees

Show step-by-step how violations are corrected after insertion or deletion.

Step 1: Identify Violation Type

→ Happens after insertion or deletion.

→ Check if the parent and uncle nodes are red or black.

Step 2: Fix Cases After Insertion

Case 1 - Uncle is Red:

→ Recolor parent and uncle black, grandparent red.

→ Move up and recheck.

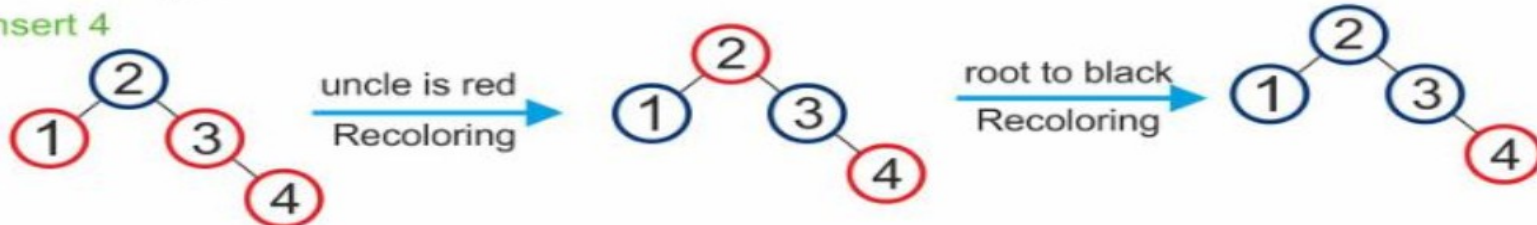
Case 2 - Uncle is Black (Triangle Case):

→ Rotate to make line alignment (e.g., left-right or right-left).

Case 3 - Uncle is Black (Line Case):

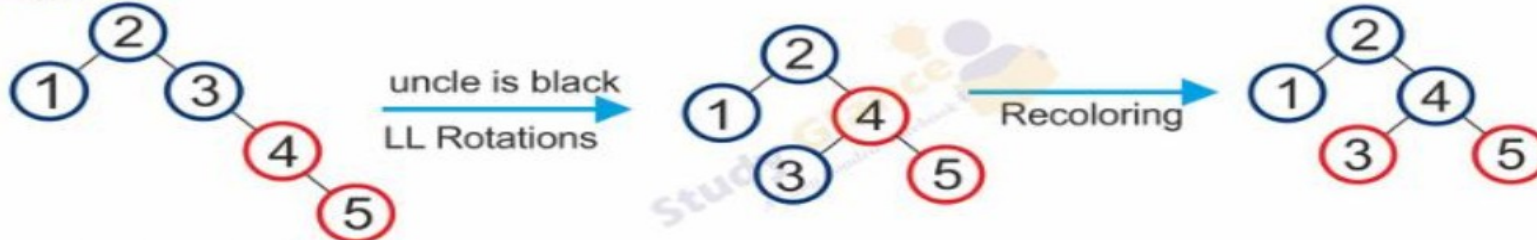
→ Rotate grandparent and recolor to restore balance

Insert 4



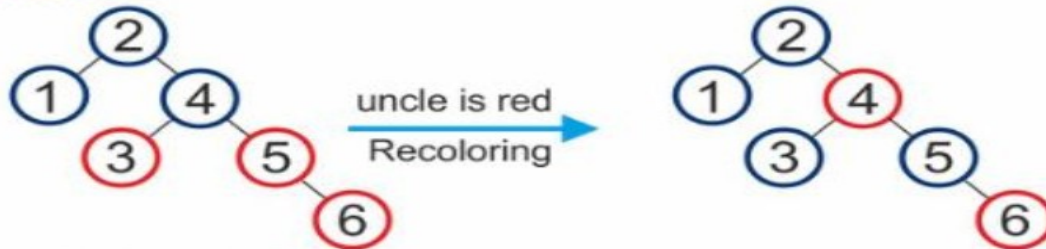
no two red nodes
can be adjacent

Insert 5



no two red nodes
can be adjacent

Insert 6



no two red nodes
can be adjacent

Deletion Algorithm in Red-Black Trees

Steps:

1. Standard BST Delete:

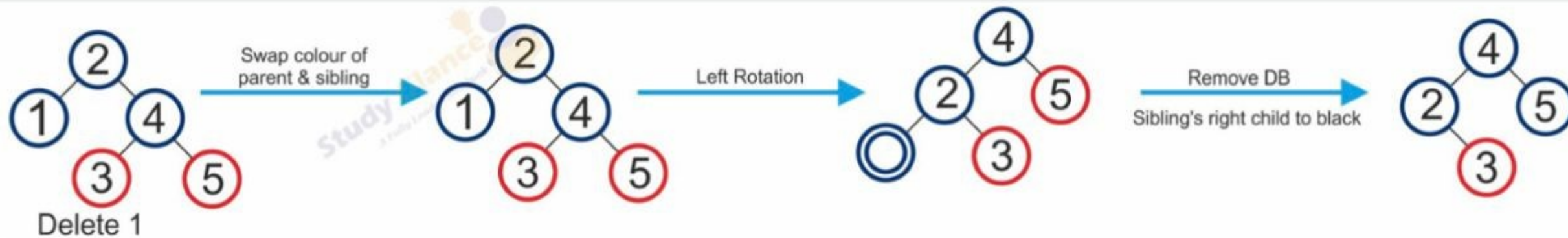
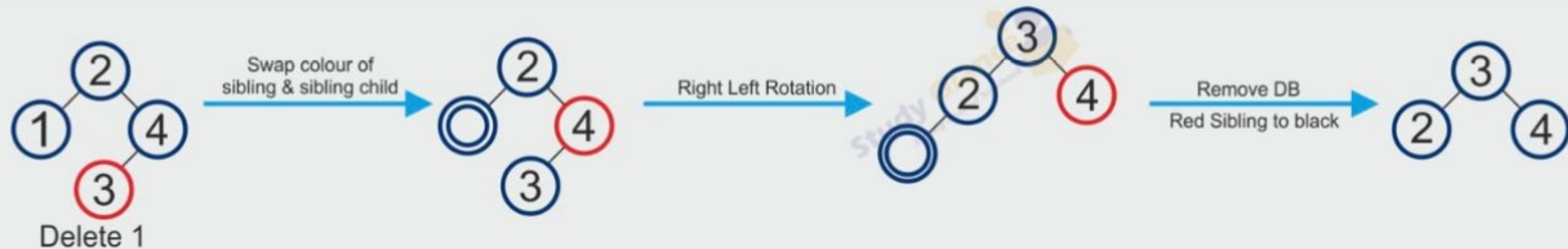
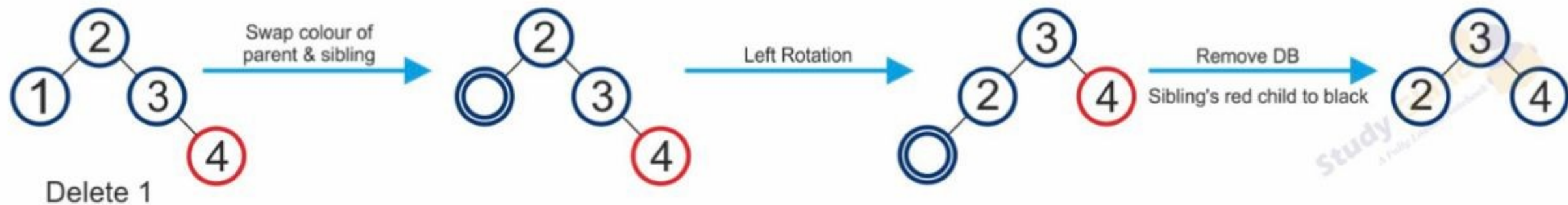
- Perform deletion as in a normal Binary Search Tree (BST). •
- If the deleted node or its replacement is black, it may cause imbalance.

2 . Fix Double-Black or Color Violations:

- When a black node is deleted, a double-black situation may occur.
- Handle by checking sibling colors and performing rotations or recolors.

3. Recolor or Rotate:

- If sibling is red → rotate around parent and recolor.
- If sibling is black → recolor and move double-black up



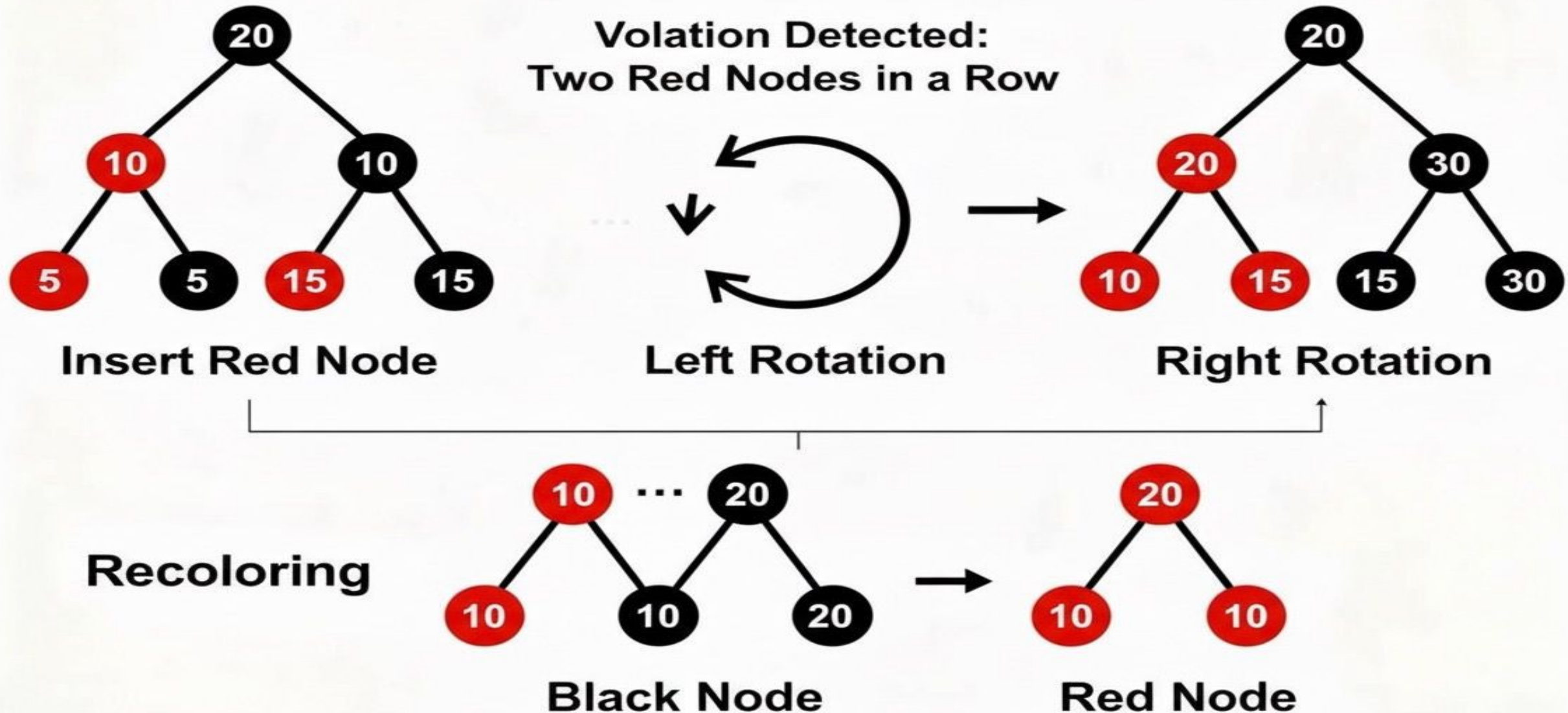
Example Insertion Sequence

- Insertion begins like a normal Binary Search Tree (BST) by comparing values and finding the correct position.
- Every newly inserted node is initially colored red to avoid unnecessary black-height violations.
- After insertion, the tree is checked for violations of Red-Black properties (e.g., red parent).
- If violations occur, the fix-up steps (recoloring or rotations) are applied to restore balance.
- The process guarantees that the tree remains approximately balanced after every insertion.

Balancing Demonstration

- Recoloring is used when the parent and uncle of the new node are red, flipping colors to maintain properties.
- Left rotation is applied when a right-leaning violation occurs to bring the structure back to balanced form.
- Right rotation is used when a left-leaning violation occurs to correct imbalance.
- Rotations ensure no red node has a red child, maintaining the self-balancing property.
- The combination of recoloring and rotations ensures consistent $O(\log n)$ height.

Red-Black Tree Insertion Balancing Process



Performance Comparison : BST VS RBT

- BST can become skewed (like a linked list), leading to worst-case $O(n)$ operations.
- RBT always maintains balanced height, ensuring $O(\log n)$ operations for insert, delete, and search.
- RBT uses extra color information to preserve structure with minimal overhead.
- BST performance heavily depends on input order; RBT performance is stable regardless of input order.
- RBT provides predictable time complexity, making it suitable for real-time and system-level operations.

Summary, Conclusion & Application

- Red-Black Trees guarantee balanced search performance through structured color rules.
- They efficiently handle large dynamic datasets that require frequent insertions and deletions.
- RBTs are widely used in implementing associative containers like maps and sets.
- Operating systems and databases use RBTs for memory management and indexing.
- Their reliable worst-case guarantees make them ideal for performance-critical applications.